



UNIVERSIDADE ESTADUAL DE SANTA CRUZ - UESC
DEPARTAMENTO DE ENGENHARIAS E COMPUTAÇÃO - DEC
ENGENHARIA QUÍMICA

INSTALAÇÃO DO ANACONDA, SPYDER E VSCODE

1 Objetivo

Este documento apresenta o procedimento recomendado para instalação e configuração das principais ferramentas computacionais que serão utilizadas durante as aulas de laboratório:

- Python;
- Anaconda e Conda;
- Spyder;
- Visual Studio Code (VSCode).

O principal objetivo é preparar um ambiente computacional estável, organizado e independente, permitindo que os programas desenvolvidos durante as aulas possam ser executados diretamente no computador do aluno, sem conflitos entre diferentes versões do Python ou bibliotecas.

A configuração proposta busca reduzir problemas comuns, tais como:

- incompatibilidade entre versões de bibliotecas;
- utilização acidental de ambientes Python incorretos;
- falhas na instalação de pacotes;
- dependência excessiva de conexão com a Internet.

Neste material, será adotado como procedimento principal o uso do **Anaconda** para criação de um ambiente Python específico para as atividades da disciplina. Esse ambiente será denominado **controle**.

Dentro desse ambiente serão instalados o Python, o Spyder e as principais bibliotecas científicas utilizadas durante as aulas, garantindo que todas as ferramentas funcionem de forma integrada e consistente.

Importante: a conexão com a Internet será necessária apenas durante o processo inicial de download e instalação dos programas e pacotes. Após a configuração completa do ambiente, os programas Python e as bibliotecas instaladas poderão ser utilizados localmente, sem necessidade de conexão permanente com a Internet.

2 Entendendo as ferramentas

Antes de iniciar a instalação, é importante compreender que Python, Anaconda, Conda, Spyder e Visual Studio Code desempenham funções diferentes e complementares no ambiente computacional.

2.1 Python

Python é a linguagem de programação utilizada para desenvolver e executar os programas.

Além da linguagem propriamente dita, Python possui um amplo conjunto de bibliotecas científicas utilizadas em aplicações de engenharia. Entre as principais bibliotecas utilizadas durante as aulas estão:

- NumPy;
- SciPy;
- Pandas;
- Matplotlib;
- SymPy.

2.2 Anaconda

Anaconda é uma distribuição Python voltada principalmente para computação científica, análise de dados e aplicações de engenharia.

Uma de suas principais vantagens é facilitar a instalação e o gerenciamento do Python e das bibliotecas científicas utilizadas em diferentes projetos.

O Anaconda também disponibiliza o **Anaconda Navigator**, uma interface gráfica que permite administrar ambientes e pacotes sem que o usuário precise necessariamente utilizar comandos no terminal.

2.3 Conda

O **Conda** é o sistema de gerenciamento de ambientes e pacotes utilizado pelo Anaconda.

Por meio do Conda é possível criar ambientes Python independentes, cada um com sua própria versão do Python e suas próprias bibliotecas. Essa separação evita que alterações realizadas em um projeto interfiram em outros projetos existentes no computador, garantindo maior estabilidade e controle sobre o ambiente de desenvolvimento.

Durante as aulas será utilizado um ambiente denominado:

trabalho

Esse ambiente concentrará o Python, o Spyder e as bibliotecas científicas necessárias para as atividades da disciplina, funcionando como um espaço isolado e seguro para execução dos programas.

2.4 Spyder

O Spyder é um ambiente de desenvolvimento integrado (IDE) desenvolvido especialmente para programação científica utilizando Python.

Entre seus principais recursos estão:

- editor de programas Python;
- console IPython;
- explorador de variáveis;
- visualização de gráficos;
- depuração de programas;
- integração com bibliotecas científicas.

Por apresentar uma interface particularmente adequada à programação científica e à análise de variáveis, o Spyder será uma das principais ferramentas computacionais utilizadas durante as aulas de laboratório.

2.5 Visual Studio Code

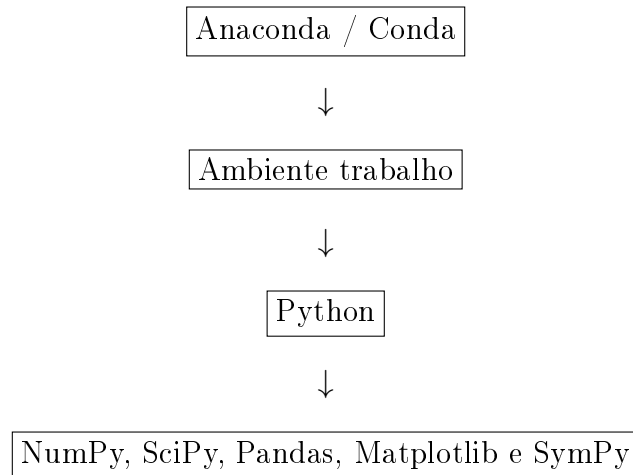
O Visual Studio Code, ou VSCode, é um editor de código que pode ser utilizado com diversas linguagens de programação.

Quando configurado para trabalhar com Python, o VSCode utiliza um interpretador Python instalado no computador. Para garantir consistência entre as ferramentas, o VSCode será configurado para utilizar o mesmo interpretador pertencente ao ambiente **trabalho** criado pelo Anaconda.

Dessa forma, Spyder e VSCode poderão utilizar o mesmo ambiente Python e as mesmas bibliotecas científicas, evitando conflitos e facilitando a manutenção do ambiente de desenvolvimento.

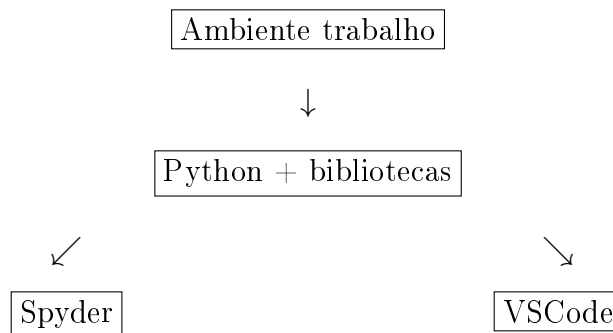
3 Estrutura recomendada

A estrutura computacional adotada neste material pode ser representada conceitualmente da seguinte maneira:



O Spyder será instalado de forma associada ao ambiente `trabalho`. O Visual Studio Code também poderá utilizar o interpretador Python pertencente a esse mesmo ambiente.

Assim, as duas ferramentas poderão trabalhar com o mesmo conjunto de bibliotecas:



Essa organização facilita a manutenção do ambiente e reduz problemas causados pela utilização de diferentes instalações do Python no mesmo computador.

4 Instalação do Anaconda

Para garantir um ambiente estável e compatível com o Spyder e as bibliotecas científicas utilizadas na disciplina, recomenda-se instalar uma versão específica do Anaconda, conhecida por sua estabilidade em computadores com diferentes configurações de hardware.

4.1 Versão recomendada

A versão recomendada para este curso é:

Anaconda3-2023.09-Windows-x86_64

Essa versão apresenta excelente compatibilidade com o Python 3.10, o Spyder e as principais bibliotecas científicas, além de funcionar adequadamente em computadores com processadores i3, i5 e i7.

4.2 Local de download

O instalador deve ser obtido no repositório oficial de versões anteriores do Anaconda:

<https://repo.anaconda.com/archive>

Importante: não utilize links diretos para o arquivo (como o endereço completo do instalador), pois alguns navegadores podem bloquear o download. Em vez disso, acesse o diretório `archive` e procure manualmente pelo arquivo:

`Anaconda3-2023.09-Windows-x86_64.exe`

4.3 Procedimento de instalação

Após realizar o download:

1. execute o instalador;
2. aceite os termos apresentados;
3. mantenha marcadas as opções:
 - **Add Anaconda to my PATH environment variable;**
 - **Register Anaconda as my default Python.**
4. clique em **Install** e aguarde a conclusão.

Ao final da instalação, o Anaconda estará pronto para uso.

5 Recomendação importante para os alunos

Para evitar problemas de compatibilidade, travamentos ou falhas na instalação, recomenda-se que todos os alunos sigam exatamente as versões e procedimentos descritos neste documento.

Instalar versões mais recentes do Anaconda ou utilizar Python 3.11/3.12 pode causar incompatibilidades com o Spyder e com bibliotecas científicas utilizadas nas aulas.

A versão recomendada é:

`Anaconda3-2023.09-Windows-x86_64`

Disponível em:

<https://repo.anaconda.com/archive>

6 Criação do ambiente de trabalho

Para garantir estabilidade e compatibilidade com o Spyder, será criado um ambiente Python específico denominado **trabalho**, utilizando a versão recomendada do Python.

Abra o **Anaconda Prompt** e execute:

```
conda create -n trabalho python=3.10
```

A versão **Python 3.10** é a mais estável para uso com o Spyder e as bibliotecas científicas utilizadas na disciplina. Versões mais recentes, como Python 3.11 ou 3.12, podem apresentar incompatibilidades e não são recomendadas.

Após a criação do ambiente, ative-o com:

```
conda activate trabalho
```

O início da linha do terminal deverá exibir:

```
(trabalho)
```

indicando que o ambiente está ativo.

7 Instalação do Spyder no ambiente controle

Com o ambiente **controle** ativo, instale o Spyder utilizando o Conda:

```
conda install spyder
```

Durante a instalação, o Conda poderá exibir avisos como:

```
Warning: using menuinst v1 shortcuts  
A newer version of conda exists
```

Esses avisos não representam erro e devem ser ignorados. Não é necessário atualizar o **conda** durante o curso.

Após a conclusão da instalação, inicie o Spyder diretamente pelo terminal:

```
spyder
```

Ao iniciar, o Spyder utilizará automaticamente o Python pertencente ao ambiente **controle**. Para confirmar isso, execute no console do Spyder:

```
import sys  
print(sys.executable)
```

O caminho exibido deverá conter:

```
anaconda3\envs\controle\python.exe
```

Importante: desativar a atualização automática do Spyder

Ao abrir o Spyder pela primeira vez, poderá aparecer uma janela oferecendo a atualização do programa. Essa atualização deve ser recusada, pois instala uma versão do Spyder fora do ambiente **controle**, causando erros de bibliotecas não encontradas.

Proceda da seguinte forma:

- Clique em **No**;
- Desmarque a opção **Check for updates at startup**.

Após isso, o Spyder funcionará corretamente dentro do ambiente **controle**.

8 Instalação das bibliotecas científicas

Após criar o ambiente **controle**, é necessário instalar as principais bibliotecas científicas que serão utilizadas durante as aulas. Todas as instalações devem ser feitas pelo *Anaconda Prompt*, garantindo que os pacotes sejam instalados dentro do ambiente correto.

8.1 Ativar o ambiente

Abra o *Anaconda Prompt* e execute:

```
conda activate controle
```

O prompt deverá exibir:

```
(controle) C:\Users\usuario>
```

8.2 Instalar as bibliotecas científicas

Com o ambiente **controle** ativo, instale as bibliotecas utilizadas na disciplina:

```
conda install numpy scipy pandas matplotlib sympy openpyxl
```

As bibliotecas instaladas são:

- **NumPy**: operações numéricas e vetoriais;
- **SciPy**: métodos numéricos e ferramentas científicas;
- **Pandas**: leitura e manipulação de tabelas e arquivos Excel;
- **Matplotlib**: geração de gráficos;
- **SymPy**: matemática simbólica;

- **openpyxl**: leitura e escrita de arquivos Excel no formato **.xlsx**.

Caso seja necessário trabalhar com arquivos Excel antigos (**.xls**), instale também:

```
conda install xlrd
```

Durante a instalação, o Conda poderá exibir avisos como:

```
A newer version of conda exists
Warning: using menuinst v1 shortcuts
```

Esses avisos não representam erros e devem ser ignorados. Não atualize o **conda** durante este processo.

8.3 Verificação das bibliotecas no Spyder

Com o ambiente **controle** ativo, inicie o Spyder:

```
spyder
```

No console do Spyder, execute:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import scipy
import sympy
import openpyxl

print("NumPy:", np.__version__)
print("Pandas:", pd.__version__)
print("SciPy:", scipy.__version__)
print("SymPy:", sympy.__version__)
print("openpyxl:", openpyxl.__version__)
```

Se todas as versões forem exibidas, as bibliotecas foram instaladas corretamente.

8.4 Teste de gráfico

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 10, 100)
y = np.sin(x)
```



```
plt.plot(x, y)
plt.title("Teste do Matplotlib")
plt.grid()
plt.show()
```

Se o gráfico for exibido, o ambiente `controle` está completamente configurado para uso nas aulas.

9 Avisos comuns durante a instalação

Durante a instalação de pacotes, alguns avisos podem ser exibidos. Eles não representam erros e podem ser ignorados:

- **Warning: using menuinst v1 shortcuts**

Indica apenas que o sistema de atalhos do Windows está utilizando uma versão anterior do menuinst.

- **A newer version of conda exists**

O Conda informa que existe uma versão mais recente disponível. Não é necessário atualizar.

- **kaleido requires Google Chrome**

Relacionado ao pacote de exportação de gráficos do Plotly. Não afeta o Spyder nem as bibliotecas utilizadas na disciplina.

Esses avisos não impedem o funcionamento do ambiente Python.

10 Desativando a atualização automática do Spyder

Ao iniciar o Spyder pela primeira vez, poderá aparecer uma janela oferecendo a atualização do programa. Essa atualização **não deve ser realizada**, pois instala uma versão do Spyder fora do ambiente `controle`, o que causa erros como:

- bibliotecas não encontradas;
- Python incorreto sendo utilizado;
- conflitos entre versões do Spyder;
- falhas ao importar pacotes instalados no ambiente.

Para evitar esses problemas, proceda da seguinte forma sempre que a janela de atualização aparecer:

1. Clique em **No**;
2. Desmarque a opção **Check for updates at startup**.

Após desativar essa opção, o Spyder não tentará mais atualizar automaticamente, garantindo que continuará funcionando corretamente dentro do ambiente **controle**.

11 Verificação da instalação do Spyder

Após iniciar o Spyder, crie um novo arquivo e digite:

```
print("Spyder instalado corretamente.")
```

Execute o programa.

A mensagem:

```
Spyder instalado corretamente.
```

deverá aparecer no console.

Em seguida, faça um teste utilizando algumas das bibliotecas científicas:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

print("NumPy:", np.__version__)
print("Pandas:", pd.__version__)

x = np.linspace(0, 10, 100)
y = np.sin(x)

plt.plot(x, y)
plt.xlabel("x")
plt.ylabel("sen(x)")
plt.title("Teste do ambiente Python")
plt.grid()
plt.show()
```

Se o programa for executado sem erros e o gráfico for apresentado, a instalação básica está funcionando corretamente.

12 Verificação do ambiente Python

É importante verificar se o programa está sendo executado pelo Python pertencente ao ambiente `trabalho`.

No Spyder, execute:

```
import sys

print(sys.executable)
print(sys.version)
```

O comando:

```
sys.executable
```

mostra o caminho completo do interpretador Python utilizado para executar o programa.

Quando o ambiente `trabalho` estiver sendo utilizado corretamente, o caminho apresentado normalmente conterá uma referência ao ambiente:

```
trabalho
```

O caminho exato pode variar conforme o sistema operacional e o local onde o Anaconda foi instalado.

13 Teste de funcionamento sem Internet

Antes da primeira aula de laboratório, recomenda-se realizar um teste simples para verificar se o ambiente está funcionando localmente.

1. Feche o Spyder.
2. Desative temporariamente a conexão Wi-Fi ou desconecte o cabo de rede.
3. Inicie novamente o Spyder.
4. Abra o programa utilizado no teste anterior.
5. Execute o programa.

Se o programa executar normalmente e o gráfico for apresentado, o ambiente está preparado para utilização local durante as aulas.

Observação:

A ausência de conexão com a Internet não impede a utilização de programas e bibliotecas que já estejam instalados no computador.

Entretanto, será necessária uma conexão com a Internet caso seja necessário:

- instalar novas bibliotecas;
- atualizar bibliotecas existentes;
- atualizar o Anaconda ou o Spyder;
- acessar arquivos ou serviços disponíveis exclusivamente na Internet.

14 Método alternativo: instalação pelo terminal

O procedimento apresentado anteriormente utiliza principalmente a interface gráfica do Anaconda Navigator por ser mais intuitiva para usuários iniciantes.

Entretanto, as mesmas operações podem ser realizadas utilizando o terminal.

Esse procedimento é particularmente útil em sistemas Linux, como Ubuntu, mas também pode ser utilizado no Windows e no macOS.

14.1 Criando o ambiente pelo terminal

Abra um terminal no qual o comando `conda` esteja disponível.

Para criar o ambiente denominado `trabalho`, execute:

```
conda create -n trabalho python
```

Quando solicitado, confirme a instalação.

Esse comando permite que o Conda selecione uma versão compatível do Python para o novo ambiente.

Em seguida, ative o ambiente:

```
conda activate trabalho
```

Após a ativação, normalmente aparecerá no início da linha do terminal:

```
(trabalho)
```

indicando que o ambiente está ativo.

14.2 Instalação das bibliotecas pelo terminal

Com o ambiente `trabalho` ativado, execute:

```
conda install numpy scipy pandas matplotlib sympy
```

Confirme a instalação quando solicitado.

14.3 Instalação do Spyder pelo terminal

Com o ambiente `trabalho` ativado, execute:

```
conda install spyder
```

Confirme a instalação e aguarde a conclusão.

Depois disso, o Spyder poderá ser iniciado pelo terminal com:

```
spyder
```

14.4 Anaconda Navigator e terminal

É importante compreender que a criação do ambiente pelo Anaconda Navigator e a criação pelo terminal não representam dois tipos diferentes de ambiente.

Nos dois casos, o gerenciamento é realizado pelo **Conda**.

A diferença está apenas na interface utilizada:

- no Anaconda Navigator, as operações são realizadas por meio de interface gráfica;
- no terminal, as mesmas operações são realizadas por meio de comandos.

Portanto, um aluno que utiliza Ubuntu, por exemplo, poderá realizar a configuração pelo terminal e obter um ambiente equivalente ao criado por outro aluno utilizando o Anaconda Navigator.

15 Verificação do ambiente pelo terminal

Com o ambiente ativado:

```
conda activate trabalho
```

execute:

```
python
```

Em seguida:

```
import sys
print(sys.executable)
print(sys.version)
```

O comando:

```
sys.executable
```

permite identificar exatamente qual interpretador Python está sendo utilizado.

Para sair do interpretador Python, execute:

```
exit()
```

16 Spyder e Visual Studio Code: papéis e diferenças

O ambiente de programação utilizado na disciplina é baseado no Python instalado dentro do ambiente `controle`. Duas IDEs podem ser utilizadas: o Spyder e o Visual Studio Code. Cada uma possui características específicas e funções distintas no processo de aprendizagem.

16.1 Spyder: IDE principal para as aulas

O Spyder é uma IDE voltada para computação científica. Ele possui console integrado, suporte nativo a gráficos e funcionamento imediato com as bibliotecas instaladas no ambiente `controle`. Por ser simples, direto e adequado ao perfil das atividades da disciplina, o Spyder será a ferramenta principal utilizada nas aulas de laboratório.

16.2 VSCode: ferramenta complementar

O Visual Studio Code é uma IDE moderna e poderosa, utilizada em projetos profissionais. No entanto, ele exige configuração manual do ambiente Python e não possui console científico integrado. Por esse motivo, o VSCode será utilizado apenas como ferramenta complementar, após o aluno dominar o uso do Spyder.

16.3 Diferença fundamental: o ambiente Python

O Spyder é instalado diretamente dentro do ambiente `controle`, garantindo que todas as bibliotecas científicas estejam disponíveis automaticamente. Já o VSCode precisa ser configurado manualmente para utilizar o Python do ambiente `controle`. Caso contrário, o VSCode utilizará o Python do ambiente base, onde as bibliotecas não estão instaladas.

16.4 Quando utilizar cada IDE

- **Spyder:** exercícios, gráficos, simulações e atividades de laboratório.
- **VSCode:** projetos maiores, organização de múltiplos arquivos e desenvolvimento profissional.

17 Instalação do Visual Studio Code

O Visual Studio Code será utilizado como ferramenta complementar durante as atividades.

Acesse o site oficial:

<https://code.visualstudio.com/>

Faça o download da versão correspondente ao sistema operacional utilizado e execute o instalador.

Depois de concluir a instalação, abra o VSCode.

18 Configuração do Python no VSCode

Para utilizar Python no VSCode, é necessário instalar a extensão correspondente.

1. Abra o VSCode.
2. Clique em **Extensions**.
3. Pesquise por:

`Python`

4. Selecione a extensão Python publicada pela Microsoft.
5. Clique em **Install**.

19 Selecionando o ambiente trabalho no VSCode

Depois de instalar a extensão Python, configure o VSCode para utilizar o mesmo ambiente criado anteriormente.

1. Pressione:

`Ctrl + Shift + P`

2. Digite:

`Python: Select Interpreter`

3. Pressione **Enter**.
4. Localize e selecione o interpretador pertencente ao ambiente:

`trabalho`

A partir desse momento, os programas executados pelo VSCode poderão utilizar as mesmas bibliotecas instaladas no ambiente `trabalho`.

Para confirmar o interpretador utilizado, execute:

```
import sys
print(sys.executable)
```

O caminho apresentado deverá corresponder ao Python pertencente ao ambiente `trabalho`.

20 Configuração recomendada do VSCode

Para facilitar a execução dos exemplos que utilizam arquivos auxiliares, como arquivos CSV, planilhas Excel e imagens, recomenda-se configurar o VSCode para executar cada programa utilizando como diretório de trabalho a pasta na qual o próprio arquivo Python está localizado.

No VSCode, pressione:

Ctrl + Shift + P

Digite:

Preferences: Open Workspace Settings (JSON)

e pressione **Enter**.

O Visual Studio Code abrirá (ou criará) o arquivo:

`.vscode/settings.json`

Adicione a seguinte configuração:

```
{
  "python.terminal.executeInFileDir": true
}
```

Essa configuração é especialmente importante quando um programa utiliza arquivos auxiliares localizados na mesma pasta do código.

Por exemplo:

```
df = pd.read_csv("dados_experimento.csv")
```

Nesse caso, o VSCode utilizará a pasta do próprio programa como diretório de trabalho, facilitando a localização do arquivo `dados_experimento.csv`.

Essa configuração também permite manter os códigos exatamente como foram desenvolvidos, sem a necessidade de inserir caminhos absolutos específicos de cada computador.

21 Problemas comuns

21.1 ModuleNotFoundError

Se aparecer uma mensagem semelhante a:

```
ModuleNotFoundError: No module named 'pandas'
```

isso normalmente significa que a biblioteca não está instalada no ambiente Python utilizado para executar o programa ou que foi selecionado um interpretador diferente.

Antes de instalar novamente qualquer pacote, verifique qual Python está sendo utilizado:

```
import sys
print(sys.executable)
```

Confirme se o caminho apresentado pertence ao ambiente `trabalho`.

21.2 FileNotFoundError

Se aparecer:

```
FileNotFoundError
```

verifique:

- se o arquivo realmente existe;
- se o nome do arquivo está correto;
- se a extensão está correta;
- se o arquivo está localizado na pasta esperada;
- se o programa está sendo executado a partir do diretório correto.

No VSCode, verifique também se a configuração apresentada anteriormente foi realizada corretamente.

21.3 PermissionError em arquivos Excel

Se um programa utilizar:

```
pd.read_excel("medicoes_laboratorio.xlsx")
```

e aparecer:

```
PermissionError: [Errno 13] Permission denied
```

verifique se a planilha está aberta no Microsoft Excel.

Em determinadas situações, o Excel pode manter o arquivo bloqueado enquanto estiver aberto para edição.

Feche a planilha no Excel e execute novamente o programa.

21.4 O programa está utilizando outro Python

Um computador pode possuir várias instalações do Python.
Por exemplo:

- Python instalado diretamente;
- Python do ambiente `base` do Anaconda;
- Python do ambiente `trabalho`;
- outros ambientes Conda.

Por esse motivo, quando surgir um problema relacionado a bibliotecas, uma das primeiras verificações deve ser:

```
import sys
print(sys.executable)
```

O resultado permite identificar exatamente qual instalação do Python está executando o programa.

22 Checklist antes da aula de laboratório

Antes de comparecer à aula, verifique:

- ☐ Anaconda instalado;
- ☐ Anaconda Navigator inicia normalmente;
- ☐ ambiente `trabalho` criado;
- ☐ Python instalado no ambiente `trabalho`;
- ☐ NumPy instalado e funcionando;
- ☐ SciPy instalado e funcionando;
- ☐ Pandas instalado e funcionando;
- ☐ Matplotlib instalado e funcionando;
- ☐ SymPy instalado e funcionando;
- ☐ Spyder instalado;
- ☐ Spyder inicia normalmente;
- ☐ programa Python executado com sucesso no Spyder;

- ☐ gráfico de teste apresentado corretamente;
- ☐ interpretador Python verificado com `sys.executable`;
- ☐ teste realizado sem conexão com a Internet;
- ☐ VSCode instalado;
- ☐ extensão Python instalada no VSCode;
- ☐ ambiente `trabalho` selecionado como interpretador no VSCode;
- ☐ configuração `python.terminal.executeInFileDir` realizada no VSCode.

O computador está preparado para as aulas de laboratório.